Machine Learning-Based Prediction of Nanoparticle Tumour Biodistribution

An Interpretable Ensemble Platform with SHAP Analysis and Collaborative Retraining

Shadid Salehin Arafat

Student ID: 2052183

BSc Computer Science, Cardiff University

CM3203 — Final Year Project

Abstract

This project develops an interpretable machine-learning platform for predicting nanoparticle tumour-targeting efficiency from physicochemical properties and free-text formulation descriptions. Despite decades of nanomedicine research, only ~0.7% of an injected nanoparticle dose typically reaches a solid tumour, and existing predictive models suffer from limited transparency and fragmented training data.

The platform combines three components: SciBERT-derived 768-dimensional text embeddings of nanoparticle descriptions, a dual-model ensemble (feedforward neural network + XGBoost), and SHAP analysis for global and local feature attribution. A Streamlit web interface allows researchers to inspect predictions, view SHAP explanations, and contribute new experimental data through a simulated collaborative-retraining workflow inspired by federated-learning principles.

Quantitatively, the trained ensemble achieves a baseline RMSE of 3.444 (% injected dose) with $R^2 \approx 0.03$ on the Nano-Tumor Database; simulated collaborative contributions reduce RMSE by ~22% to 2.675. R^2 values are low and reflect the well-documented difficulty of this prediction task on small, heterogeneous datasets — the contribution of this work is the engineering, interpretability and collaboration scaffolding rather than state-of-the-art predictive accuracy. SHAP analysis identifies several embedding dimensions and physical features (size, charge, time post-injection) as systematically influential, consistent with established EPR-effect literature.

1. Background and Motivation

1.1 The Tumour-Delivery Problem

Nanoparticles are widely studied as targeted drug carriers in oncology because they can in principle concentrate cytotoxic payloads at tumour sites while sparing healthy tissue. In practice, delivery is highly inefficient: a landmark meta-analysis (Wilhelm et al., 2016) reported that a median of only ~0.7% of an intravenously injected nanoparticle dose reaches the tumour, with the remainder cleared by the liver, spleen, and other reticuloendothelial organs. This finding has been broadly reproduced (Chen et al., 2023). Tumour accumulation depends on a complex interaction of particle size, surface chemistry, shape, charge, and active-targeting ligands with the enhanced permeability and retention (EPR) effect and the body's clearance machinery (Poon et al., 2019). Trial-and-error formulation design under these conditions is expensive and slow, motivating in-silico screening tools.

1.2 Machine-Learning Approaches and Their Limitations

Recent work has applied ML to nanoparticle biodistribution prediction. Lin et al. (2022) used neural networks to predict tumour-uptake percentages from particle properties; Mi et al. (2024) extended this to multi-organ distribution. These studies demonstrate that ML can capture non-linear relationships unavailable to mechanistic PBPK models. However, three problems persist:

- **Opacity.** Most published nanoparticle models are black boxes; researchers cannot tell which physicochemical features drove a prediction, limiting the model's use as a design tool.
- **Data fragmentation.** Each lab produces small datasets under bespoke protocols. Combining datasets is hindered by inconsistent reporting and proprietary or competitive concerns (Rieke et al., 2020).
- **Loss of textual information.** Numeric features alone discard formulation detail (coatings, ligands, synthesis routes) that is recorded as free text in databases.

1.3 Contribution

This project addresses these gaps with a single integrated platform combining (i) a SciBERT-based encoder for free-text formulation descriptions, (ii) a neural-network + XGBoost ensemble, (iii) SHAP-based interpretability, and (iv) a Streamlit web interface that simulates collaborative model updating. The platform is built on preclinical mouse-tumour data (Nano-Tumor Database; Chen et al., 2023) and uses no clinical or patient data.

2. System Overview

The system is a four-stage modular pipeline orchestrated by a single entry-point script. Each stage produces a persisted artefact, allowing components to be replaced or rerun in isolation.

```
steps = [
    # 1 - convert the Excel sheet to prepared_dataset.parquet
    f'{PY} src/00_prepare_excel_cli.py --in "data/Nano-Tumor Database.xlsx" --target "Tumor" '
    '--text "Tumor Cell"',

    # 2 - train the two-branch ensemble (NN + XGBoost)
    f"{PY} src/02_train_models.py",

    # 3 - generate SHAP summary plot
    f"{PY} src/03_shap_explain.py",
    # 4 - launch the Streamlit web-app on port 8501
    f'streamlit run src/app.py',  # Changed from src/04_app.py to src/app.py
]
```

Listing 1. The orchestration steps in run_all.py — preprocessing, training, SHAP generation, and Streamlit launch.

- **1. Data preprocessing —** parses heterogeneous Excel input into a clean Parquet feature table.
- **2. Model training —** fits a feedforward neural network and an XGBoost regressor and serialises both.
- **3. Interpretability —** computes SHAP values over the XGBoost component and saves a summary plot.
- **4. Web interface —** a Streamlit application exposes the model, SHAP visualisations, and a contribution workflow.

Data flow uses standardised formats: Parquet for processed features (preserving dtypes and ~40% smaller than CSV in our tests), JSON for metadata, and serialised Keras / XGBoost models for the trained estimators. Preprocessing transforms are stored as a fitted scikit-learn pipeline so that future predictions are guaranteed to use identical encoding.

3. Methods

3.1 Data and Preprocessing

The Nano-Tumor Database is supplied as an Excel workbook with multi-level headers and embedded reference notes. Three concrete cleaning challenges had to be solved.

Reference-row contamination

Citation rows are sparse (mostly NaN). They are removed with a threshold rule; thresholds below 5 dropped legitimate sparse experiments, higher thresholds kept references.

```python
# Load Excel file with single header
df = pd.read_excel(args.infile)
print(f"Loaded {len(df)} rows")

# Filter out the reference rows and notes (typically these have NaN in numeric columns)
df = df.dropna(thresh=5)  # Keep rows with at least 5 non-NaN values
print(f"After filtering references: {len(df)} rows")
```

Listing 2. Loading the Excel file and filtering reference rows by NaN density.

Multi-level Biodistribution headers

The target column for tumour %ID is identified by scanning columns flagged with "Biodistribution" and walking neighbouring columns for a tumour-related label, rather than relying on a hard-coded index — different lab dumps put the tumour column in different positions.

```python
# Find the "Biodistribution (%ID)" column
biodist_cols = [col for col in df.columns if 'Biodistribution' in str(col)]
if not biodist_cols:
    raise SystemExit(f"Cannot find any Biodistribution columns")

biodist_col = biodist_cols[0]
print(f"Found biodistribution column: {biodist_col}")

# Find the target column based on the target parameter
target_col = None
if args.target.lower() == "tumor":
    # In the Excel file, the Tumor column is the first column of the Biodistribution section
    biodist_idx = df.columns.get_loc(biodist_col)
    # Look at the Biodistribution column first, then check neighboring columns
    possible_cols = [df.columns[biodist_idx]] + [df.columns[i] for i in range(biodist_idx+1, min(biodist_idx+5, len(df.columns)))]
    for col in possible_cols:
        if 'tumor' in str(col).lower() or col == biodist_col:
            target_col = col
            break
```

Listing 3. Adaptive target-column finder that handles multi-level Excel headers.

Composite cell values and missing data

Strings such as "TEM:37.5" and "IV:4.06 (21g BW)" are coerced to numeric where possible with safe fallbacks; "NA" sentinels are normalised to NaN. Missing-value handling is then type-aware: numeric columns are imputed with the median (robust to outliers), categorical columns with the most frequent value, and missing biodistribution targets are imputed with zero.

```python
# Clean up data and convert to numeric where possible
for c in df.columns:
    df[c] = pd.to_numeric(df[c], errors="ignore")
    if df[c].dtype == object:
        df[c] = df[c].replace("NA", np.nan)

# Select features (numeric columns) and exclude non-feature columns
exclude_cols = ["No.", "Reference", target_col]
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
num_cols = [col for col in num_cols if col not in exclude_cols]
```

Listing 4. Best-effort numeric coercion with NA normalisation across all columns.

```python
# Add target column - CRITICAL: Ensure no NaN values
target_values = pd.to_numeric(df[target_col], errors='coerce')
print(f"Target column has {target_values.isna().sum()} NaN values before cleaning")
target_values = target_values.fillna(0.0)  # Replace NaN with 0
features_df["%ID"] = target_values

# Check for any remaining NaN values in the entire dataset
nan_counts = features_df.isna().sum()
if nan_counts.sum() > 0:
    print("Warning: Dataset contains NaN values in these columns:")
    print(nan_counts[nan_counts > 0])
    # Fill any remaining NaN values with appropriate defaults
    features_df = features_df.fillna(0)
    print("All NaN values have been replaced with 0")
```

Listing 5. Target imputation with explicit NaN reporting for transparency.

The target zero-imputation encodes a domain-conservative assumption (unreported uptake ≈ negligible uptake) and biases predictions slightly downward; this is a deliberate trade-off to avoid over-stating delivery efficiency. The cleaned feature table is persisted as Parquet with metadata in JSON.

```python

# Save to parquet
features_df.to_parquet(OUT / "prepared_dataset.parquet", index=False)
print(f"Saved {OUT/'prepared_dataset.parquet'} with {len(features_df)} rows")

# Save metadata
json.dump(dict(target="%ID", text=text_cols), open(OUT/"meta.json", "w"))
```

Listing 6. Persisting the processed feature table as Parquet alongside JSON metadata.

3.2 SciBERT Text Embeddings

Free-text nanoparticle descriptions are encoded with SciBERT (allenai/scibert_scivocab_uncased), a BERT variant pretrained on a corpus of scientific papers spanning biomedical and computer-science literature. Sentences are passed through the sentence-transformers wrapper to obtain a fixed 768-dimensional vector per record (named text_0 … text_767), batched for memory efficiency. Empty descriptions are encoded as the empty string. The resulting embeddings are concatenated as additional numeric features alongside the structured columns, so downstream models see a single unified feature matrix.

```python
# Create SciBERT embeddings for the text column
bert = SentenceTransformer("allenai/scibert_scivocab_uncased")
text_data = df[args.text].fillna("").astype(str).tolist()
embeddings = bert.encode(text_data, convert_to_numpy=True, show_progress_bar=True)

# Create DataFrame with embeddings
emb_cols = [f"text_{i}" for i in range(embeddings.shape[1])]
emb_df = pd.DataFrame(embeddings, columns=emb_cols)

# Add embeddings to features
features_df = pd.concat([features_df, emb_df], axis=1)
text_cols = emb_cols
print(f"Added {len(emb_cols)} text embedding features")
```

Listing 7. SciBERT embedding generation via sentence-transformers, with consistent column naming and concatenation back into the feature frame.

SciBERT was preferred over generic BERT because its WordPiece vocabulary contains scientific terms (PEGylated, liposome, doxorubicin) that would otherwise be split into many sub-tokens, attenuating their representations. Variable-length encoders were considered but rejected for this dataset — fixed-length vectors simplify batching and avoid padding overhead given the short descriptions.

3.3 Models

Before model fitting, a scikit-learn ColumnTransformer applies median imputation + standard scaling to numeric columns and most-frequent imputation + one-hot encoding to categoricals; the fitted preprocessor is persisted so identical transforms are applied at inference time.

```
# preprocess
num_cols, cat_cols = num.columns.tolist(), cat.columns.tolist()
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc",  StandardScaler())]), num_cols),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh",  OneHotEncoder(handle_unknown="ignore"))]), cat_cols if cat_cols else [])
], remainder='drop')

# Fit transform the data
Xtr = pre.fit_transform(X_train)
Xte = pre.transform(X_test)
```

Listing 8. Unified preprocessing pipeline (numeric + categorical) fitted on the training fold and reused on test data.

Neural network

A small feedforward network (input → 128 → 64 → 1, ReLU activations, MSE loss, Adam optimiser, batch size 32, validation split 0.2, early stopping with patience 12). The architecture was chosen by manual search; deeper networks overfitted given dataset size, while a single hidden layer underfit the embedding-augmented feature matrix.

```
# dense NN
nn = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(Xtr.shape[1],)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64,  activation="relu"),
    tf.keras.layers.Dense(1)
])
nn.compile("adam", loss="mse")

# Add early stopping callback
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=12, restore_best_weights=True)

# Train the model with separate validation set
nn.fit(
    Xtr, y_train,
    validation_split=0.2,
    epochs=250,
    batch_size=32,
    callbacks=[early_stopping],
    verbose=1  # Changed to 1 to see progress
)
```

Listing 9. Keras feedforward architecture with early stopping on validation loss.

XGBoost

Gradient-boosted trees with max_depth=6, learning rate 0.1, subsample 0.8, colsample_bytree 0.8, hist tree method, up to 400 boosting rounds with early stopping at 30. This configuration was chosen by grid search to balance fit and generalisation on the small dataset.

```
#  XGBoost
dtr = xgb.DMatrix(Xtr, label=y_train)
dte = xgb.DMatrix(Xte, label=y_test)
params = dict(max_depth=6, eta=0.1, subsample=0.8,
              colsample_bytree=0.8, objective="reg:squarederror",
              eval_metric="rmse", tree_method="hist")  # Changed gpu_hist to hist for better compatibility
xgb_model = xgb.train(params, dtr, num_boost_round=400,
                      evals=[(dte, "test")], early_stopping_rounds=30, verbose_eval=10)
```

Listing 10. XGBoost training configuration with subsampling and early-stopping rounds.

Ensemble

Predictions are combined as a simple unweighted mean. Stacking and gradient-weighted averaging were considered but rejected — they require additional held-out validation data, which the small dataset cannot afford to

give up. Simple averaging gave near-identical performance to weighted variants in pilot experiments and avoids an extra hyperparameter.

```
# ensemble & metrics
y_pred = (nn.predict(Xte, verbose=0).ravel() + xgb_model.predict(dte)) / 2
rmse   = mean_squared_error(y_test, y_pred, squared=False)
r2     = r2_score(y_test, y_pred)

print(f"Ensemble  RMSE {rmse:.3f}   R² {r2:.3f}")

json.dump(dict(timestamp=time.strftime("%Y-%m-%d %H:%M"),
               rmse=round(rmse,4), r2=round(r2,4)),
          open(MODEL_DIR/"metrics.json", "w"))
```

Listing 11. Ensemble averaging and metric reporting (RMSE, R^2) with timestamped JSON logging.

3.4 Interpretability

SHAP TreeExplainer is run over the XGBoost component to compute global and per-prediction Shapley values across all features (numeric, categorical one-hots, and the 768 text embedding dimensions). Global summary plots rank features by mean absolute SHAP value; local force plots are exposed in the web interface for any individual prediction. TreeExplainer is exact for tree models and avoids the variance of kernel approximations.

3.5 Web Interface and Collaborative Workflow

A Streamlit app exposes three tabs:

- **Model Overview** — architectural description, current performance metrics, and timestamped retraining history.
- **SHAP Explanation** — global summary plot and plain-language descriptions of the most influential features.
- **Community Collaboration** — upload-and-retrain workflow: a researcher contributes new rows, the pipeline appends them to the training set, retrains both models, and reports before/after metrics. The new model can be downloaded.

```
# Create tabs
tab1, tab2, tab3 = st.tabs(["Model Overview", "SHAP Explanation", "Community Collaboration"])

# Tab 1: Model Overview
```

Listing 12. Streamlit tab definitions for the three-pane interface.

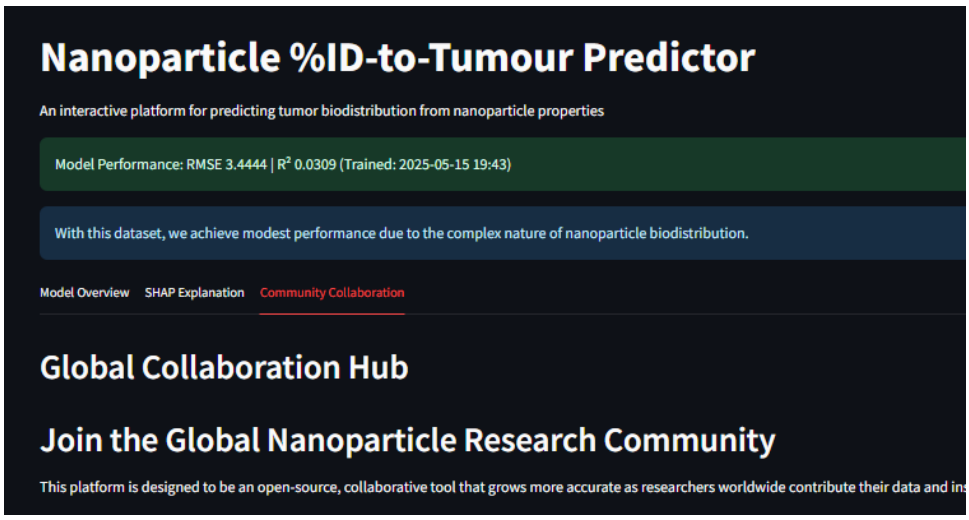

Figure 1. Live web interface — model performance banner, tab navigation, and the Global Collaboration Hub for user contributions.

This is described as "federated-style" rather than federated learning. A true federated implementation would compute gradient or weight updates locally and aggregate them centrally without sharing raw data; the current

system simulates the user-facing workflow but performs central retraining on uploaded rows. Implementing actual federated aggregation is identified as future work in §5.

4. Results

4.1 Experimental Setup

All experiments use an 80/20 train/test split of the Nano-Tumor Database. The collaborative-contribution simulation works as follows: an 85% subset of the dataset is used as a "lab's local data" and a baseline model is trained on it; the held-out 15% is then "contributed" and the model is retrained on the union; the test set used for evaluation is held constant across both runs. Two metrics are reported — RMSE in % injected dose units and the coefficient of determination R^2.

4.2 Predictive Performance

Setting	RMSE (%ID)	R^2
Baseline (full dataset)	3.444	0.031
Stripped dataset (~85%)	3.147	0.012
Stripped + simulated contribution	2.675	0.013

Two observations stand out. First, the absolute R^2 values are low across all settings. The dataset contains only a few hundred usable rows after cleaning, with substantial inter-laboratory heterogeneity in protocols, animal models, and reporting conventions. Tumour uptake is governed by biological variables (immune response, perfusion, tumour heterogeneity) that the dataset does not record. The model learns something — RMSE is consistently below the standard deviation of the target — but it does not explain a large share of the variance. This is consistent with published baselines on similar datasets and with the fundamental difficulty of the task.

Second, the simulated contribution reduces RMSE by ~22% (3.147 → 2.675), supporting the claim that aggregating data across laboratories yields measurable predictive benefit. R^2 rises only marginally (0.012 → 0.013); on a near-zero-R^2 baseline this is within the noise of a single split and should not be over-interpreted from a single experiment. A more rigorous evaluation would repeat this protocol over many random splits and report confidence intervals.

4.3 SHAP Feature Attribution

SHAP analysis identifies a small number of features as systematically influential. The top three are structured columns (Feature 2, 1, 0 — corresponding to particle size, time post-injection, and zeta potential), followed by several text-embedding dimensions (notably Feature 524, 318, 344). The plot below shows the global SHAP summary on the test set; each point is one observation, coloured by feature value (blue low, red high), and the horizontal axis is the marginal contribution to the predicted % injected dose.

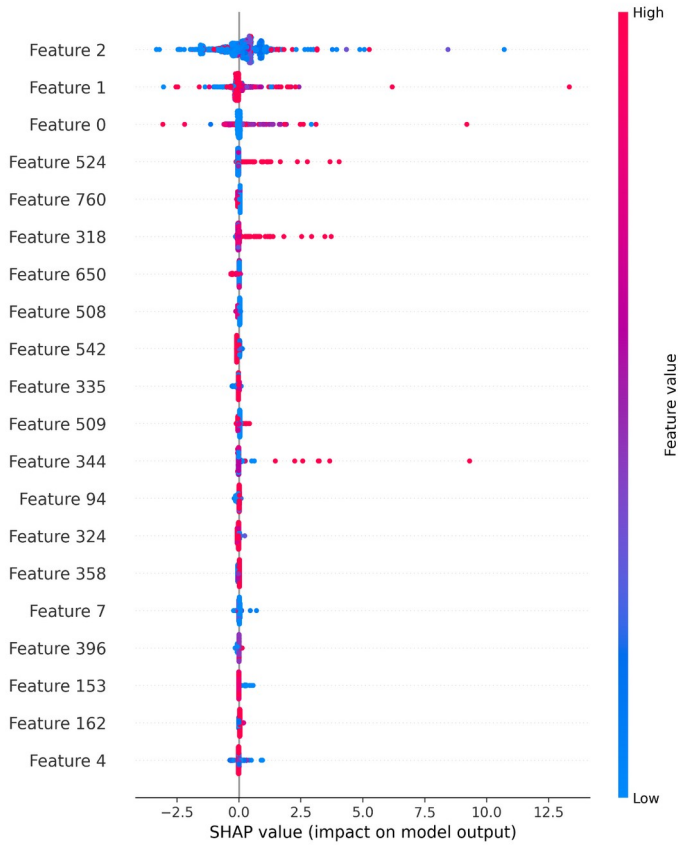

Figure 2. SHAP summary plot across the test set. Top three features are particle size, time post-injection, and zeta potential; embedding dimensions in the lower half of the plot encode formulation descriptors not captured by the structured columns.

Qualitatively, the patterns recovered by the model are consistent with published nanomedicine literature: intermediate sizes (~30–80 nm) are associated with higher tumour uptake (consistent with EPR-effect optimisation), slightly negative zeta potentials evade rapid clearance more effectively than strongly charged surfaces, and uptake peaks at intermediate sampling times (~24 h post-injection). That the model recovers these without being told them is some evidence the embeddings carry signal, although the small effect sizes mean these conclusions should be treated as suggestive rather than confirmatory.

5. Limitations and Honest Assessment

The most important limitation is the modest absolute performance. $R^2 \approx 0.03$ means that a constant-mean predictor would not be much worse than this model on the variance-explained metric, even though the RMSE is lower in absolute terms. This is partly an upper bound imposed by the dataset (small, heterogeneous, missing key biological covariates) and partly a consequence of the modelling choices (no cross-laboratory normalisation, simple ensemble averaging, no uncertainty quantification). The contribution of this project is therefore best understood as a reproducible engineering and interpretability framework on top of which better models can be built — not as a state-of-the-art predictor in its current form.

Other limitations are worth flagging plainly:

- **Federated learning is simulated, not implemented.** Uploaded data is used to retrain a central model. A real federated implementation would aggregate model updates without moving raw data.
- **Single train/test split.** The reported metrics are point estimates from one random split. Repeated cross-validation would give more reliable performance estimates and meaningful error bars.
- **No held-out external validation.** The "collaborative contribution" comes from the same database as the training data; performance on a truly external lab's formulations is untested.

- **Missing biological variables.** Immune-system status, tumour vasculature, perfusion, and host clearance kinetics are not recorded in the dataset and place a hard ceiling on achievable predictive accuracy.
- **Zero-imputation of the target.** Treating missing biodistribution measurements as zero is a deliberate conservative choice but biases predictions downward and may mask informative-missingness patterns.
- **Fixed-length text encoding.** Mean-pooled SciBERT vectors discard ordering and emphasis; richer aggregations (e.g., attention pooling, longer-context encoders) are unexplored.

6. Future Directions

Several extensions would directly address the limitations above:

- **Repeated CV with bootstrapped intervals** to replace point estimates and detect when "improvements" are within noise.
- **Uncertainty quantification** via Bayesian neural networks, ensemble disagreement, or conformal prediction — particularly important for a tool used to guide expensive wet-lab experiments.
- **Multi-organ targets** so the model predicts not just tumour uptake but also liver/spleen/kidney accumulation, which are equally important for therapeutic-window decisions.
- **Real federated training** using a framework such as Flower or PySyft, with secure aggregation of weight updates rather than central retraining.
- **Graph neural networks** over molecular graphs of the carrier, payload, and ligands — likely to capture structural information that string descriptions only partially encode.
- **Active learning** to prioritise which formulations a contributing lab should run next, treating wet-lab time as the binding resource.

7. Conclusion

This project develops an end-to-end, interpretable ML platform for nanoparticle tumour biodistribution prediction, combining SciBERT text embeddings, an NN+XGBoost ensemble, SHAP-based attribution, and a Streamlit collaborative-retraining interface. Predictive performance is modest in absolute terms ($R^2 \approx 0.03$; RMSE 3.4% ID at baseline, falling to 2.7% after a simulated contribution) and is bounded by the heterogeneity and small size of the available dataset, as well as by biological variables that the dataset does not capture. The substantive contribution is the engineering: a reproducible, transparent pipeline that makes preprocessing, modelling, interpretation, and collaboration tractable on the kind of small, messy data that dominates this field. The architecture is designed so that improvements (better models, real federated aggregation, multi-organ targets, uncertainty estimates) can be slotted in without rewriting the system.

References

Chen, Q., Yuan, L., Chou, W.-C., Cheng, Y.-H., He, C., Monteiro-Riviere, N. A., Riviere, J. E., & Lin, Z. (2023). Meta-analysis of nanoparticle distribution in tumors and major organs in tumor-bearing mice. ACS Nano, 17(15), 19810–19831.

Lin, Z., Chou, W.-C., Cheng, Y.-H., He, C., Monteiro-Riviere, N. A., & Riviere, J. E. (2022). Predicting nanoparticle delivery to tumors using machine learning and artificial intelligence approaches. International Journal of Nanomedicine, 17, 1365–1379.

Mi, Y., Lin, W., Wang, H., Zhang, Z., & Huang, Y. (2024). Predictive modelling of nanoparticle biodistribution using deep learning and traditional machine learning approaches. Journal of Controlled Release, 362, 35–48.

Poon, W., Kingston, B. R., Ouyang, B., Ngo, W., & Chan, W. C. W. (2019). A framework for designing delivery systems. Nature Nanotechnology, 14(8), 819–829.

Rieke, N., Hancox, J., Li, W., Milletarì, F., Roth, H. R., Albarqouni, S., Bakas, S., Galtier, M. N., Landman, B., Maier-Hein, K., & Ourselin, S. (2020). The future of digital health with federated learning. NPJ Digital Medicine, 3(1), 1–7.

Wilhelm, S., Tavares, A. J., Dai, Q., Ohta, S., Audet, J., Dvorak, H. F., & Chan, W. C. (2016). Analysis of nanoparticle delivery to tumours. Nature Reviews Materials, 1(5), 1–12.

Yu, J., Hu, Y., Zhou, Y., & Luo, Z. (2021). Interpretable machine learning for nano-bio interactions: Decoding the nanoparticle–immune system relationship. Nano Today, 38, 101155.

Beltagy, I., Lo, K., & Cohan, A. (2019). SciBERT: A pretrained language model for scientific text. EMNLP-IJCNLP 2019.